

Technical Stack & Dependencies Guide

Official Architecture Documentation - Smart Sheets AI Pipeline

Developer: Deivis Azeredo • Role: AI Automation Specialist & Prompt Engineer • Date: May 2026

1. Architecture Overview

The **Smart Sheets AI Pipeline** is a robust automation solution developed using the *AI-Assisted Coding* (Vibecoding) methodology. The ecosystem is designed to extract, validate, sanitize, and enrich structured text data in spreadsheets, eliminating operational bottlenecks, grammatical errors, and mitigating plagiarism/duplicate content.

2. Core Language: Python

Python 3.10+ acts as the central Core Execution Engine of the entire application. It was selected for its efficiency in local script management, data file manipulation, and native, lightweight connectivity with ultra-low latency Artificial Intelligence APIs.

3. AI Models Orchestration (LLMs)

The cognitive intelligence and logic of the pipeline are decentralized and built through the combination of three leading AI ecosystems:

Technology / Model	Ecosystem Role	Strategic Description
Groq API / Llama 3.3	Production / Execution	Responsible for real-time processing of spreadsheet texts. Selected due to its unmatched inference speed and analytical precision.
Anthropic Claude	Engineering / Architecture	Acts as the assisted Senior Software Architect, refining system prompts and structuring complex source code logic.
Google Gemini	Engineering / Architecture	Utilized for feature mapping, business rule validation, and accelerated infrastructure script generation.

4. Libraries and Modules Used

To ensure the modular and portable operation of the software, native and third-party packages were integrated:

4.1 openpyxl

A third-party library used to read and write directly to Excel files (.xlsx) located in the same directory as the script. It guarantees absolute portability without relying on complex cloud authentications for third parties testing the portfolio.

4.2 groq (Official SDK)

The official Python client that manages HTTPS requests and secure communication (data transport layer) with Groq Cloud servers.

4.3 json (Native Python)

An internal Python utility module used for data manipulation, reading, and structuring configurations and data objects (essential in integration architectures and handling API responses).

4.4 time & os (Native Python)

Native modules for operating system path management (ensuring the script finds the spreadsheet wherever it is executed) and request rate management (Rate Limits) using the `time.sleep()` command.

5. Standardized Installation File (requirements.txt)

In the professional Python ecosystem, external dependencies that are not part of the standard language must be listed in a file called `requirements.txt`. This allows any recruiter or client to install all tools with a single terminal command.

Market Perspective: Including this file in your GitHub repository demonstrates a clear understanding of the workflow standard used by professional engineering teams in the international market.

The exact content of the file generated for the repository is:

```
openpyxl>=3.1.0
groq>=0.4.0
```